

KONGU ENGINEERING COLLEGE

SMART FLOOD LEVEL MONITORING SYSTEM USING ULTRASONIC SENSOR

MECHATRONICS AND IOT

EXCERSICE – 06

HEMAVARSHINI S - 24MER017

KAVIN KUMARAN S - 24MER023

KOWSIKA K - 24MER026

VISHNUPRASATH B - 24MER062



KONGU ENGINEERING COLLEGE

— TRANSFORM YOURSELF —

SMART FLOOD LEVEL MONITORING SYSTEM USING ULTRASONIC SENSOR

1. Project Overview

Introduction

Flooding is one of the most frequent and damaging natural disasters, and timely awareness of a rising water level is critical for effective disaster management. Conventional water-level monitoring is often manual, relying on periodic physical inspection of rivers, canals, or reservoirs, which does not provide continuous, real-time awareness of rising water conditions.

The Smart Flood Level Monitoring System is an IoT-based system built around an ESP32 microcontroller that continuously monitors the water level using a non-contact HC-SR04 ultrasonic sensor. The ultrasonic sensor measures the distance between itself and the surface of the water; as the water rises, this distance decreases, allowing the ESP32 to calculate the current water level in real time.

The measured water level and system status are displayed locally on an OLED display and simultaneously uploaded to the ThingSpeak IoT cloud platform over Wi-Fi. When the water level rises above a predefined flood threshold, the ESP32 immediately activates a red LED and a buzzer to warn nearby personnel, while a green LED indicates normal, safe conditions.

The main objective is to provide continuous, automated flood-level monitoring with an immediate local alert and remote cloud visibility, supporting faster response in disaster management applications.

Main Features

- Continuous, non-contact water-level measurement using an ultrasonic sensor.
- Automatic threshold-based flood detection.
- Local visual alerts using green (safe) and red (flood warning) LEDs.
- Audible buzzer alarm during critical flood conditions.
- Real-time water level and status display on an OLED screen.
- Cloud data logging and remote monitoring through ThingSpeak.
- Wi-Fi based IoT connectivity using ESP32.
- Low-cost, expandable architecture suitable for disaster management applications.

2. Problem Statement

Flood-prone regions such as riverbanks, low-lying urban areas, dams, and canals require continuous monitoring of water levels to enable timely evacuation and disaster response. Conventional monitoring methods depend on manual observation or fixed-interval checks, which cannot provide continuous, real-time information.

This can result in:

- Delayed detection of rapidly rising water levels.

- Late warning to nearby residents and authorities.
- Increased risk to life and property.
- Difficulty coordinating timely evacuation.
- Lack of historical water-level data for disaster-management planning.
- Dependence on continuous human observation, which is impractical for remote areas.

Therefore, an automated system is required to continuously measure the water level and provide an immediate local and remote alert as soon as the level crosses a critical flood threshold.

3. Proposed Solution

The proposed system uses an ultrasonic sensor and a microcontroller to automate flood-level monitoring. The HC-SR04 ultrasonic sensor is mounted above the water body and measures the distance to the water surface by sending an ultrasonic pulse and timing its reflection. The ESP32 converts this distance into a water-level value based on the known mounting height.

The ESP32 compares the calculated water level against a predefined flood threshold to determine the safety status.

When the water level rises above the threshold:

Ultrasonic Sensor → ESP32 → Status = FLOOD WARNING → Red LED ON + Buzzer ON

When the water level remains within the safe range:

Ultrasonic Sensor → ESP32 → Status = SAFE → Green LED ON, Buzzer

In parallel with the local alert logic, the ESP32 continuously updates the OLED display with the current water level and status, and uploads the readings to the ThingSpeak cloud platform, enabling both immediate local warning and remote monitoring for disaster management.

4. System Architecture

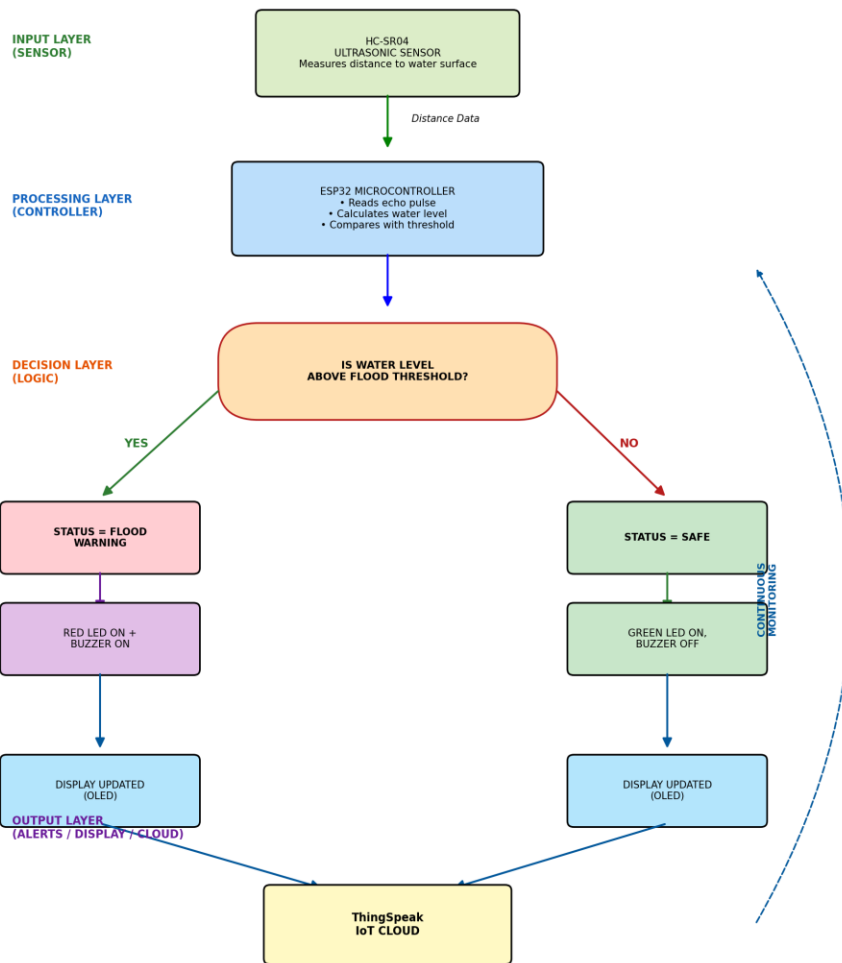
The overall architecture of the system is ,

System Flow:

Ultrasonic Sensor → ESP32 → Threshold Comparison → LED/Buzzer Alert + OLED Display → ThingSpeak Cloud

SMART FLOOD LEVEL MONITORING SYSTEM

SYSTEM ARCHITECTURE FLOWCHART



5. Circuit Table

Component	Pin	ESP32
HC-SR04 Ultrasonic Sensor	TRIG	GPIO 5
HC-SR04 Ultrasonic Sensor	ECHO	GPIO 18
HC-SR04 Ultrasonic Sensor	VCC / GND	5V / GND
Green LED (Safe Indicator)	Anode	GPIO 2
Red LED (Flood Warning)	Anode	GPIO 4
Buzzer	SIG (+)	GPIO 15
Buzzer	GND (-)	GND
OLED Display (I2C)	SDA	GPIO 21
OLED Display (I2C)	SCL	GPIO 22
OLED Display (I2C)	VCC / GND	3V3 / GND

Power Supply

The ESP32 can be powered through USB or a regulated 5V supply. The HC-SR04 requires a 5V supply on VCC while its TRIG/ECHO logic is read directly by the ESP32. The OLED display, LEDs and buzzer are driven from the ESP32's 3V3/5V rails and GPIO pins, with all grounds tied to a common ESP32 GND.

6. Circuit Design

The circuit consists of four major sections:

6.1 ESP32 Microcontroller

The ESP32 is the main controller. It triggers the ultrasonic sensor, measures the echo pulse to compute distance and water level, drives the LEDs, buzzer and OLED display, and connects to Wi-Fi to upload data to ThingSpeak.

6.2 HC-SR04 Ultrasonic Sensor

The ultrasonic sensor is mounted facing downward above the water body. It is connected to two digital pins — one to trigger the pulse and one to read the returning echo.

ESP32 GPIO 5 → HC-SR04 TRIG
HC-SR04 ECHO → ESP32 GPIO 18

6.3 Green and Red LED Indicators

The green LED indicates a safe water level, and the red LED indicates a flood warning condition.

ESP32 GPIO 2 → Green LED | ESP32 GPIO 4 → Red LED

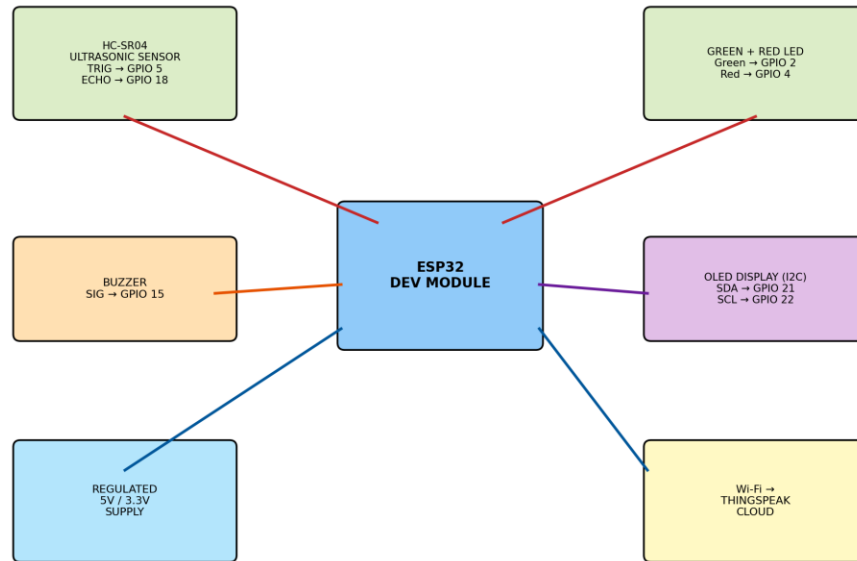
6.4 Buzzer and OLED Display

The buzzer provides an immediate audible alert during flood conditions, and the OLED display continuously shows the measured water level and current status.

ESP32 GPIO 15 → Buzzer SIG | ESP32 GPIO 21/22 → OLED SDA/SCL

CIRCUIT DESIGN:

FLOOD MONITORING SYSTEM - CIRCUIT CONNECTION DIAGRAM

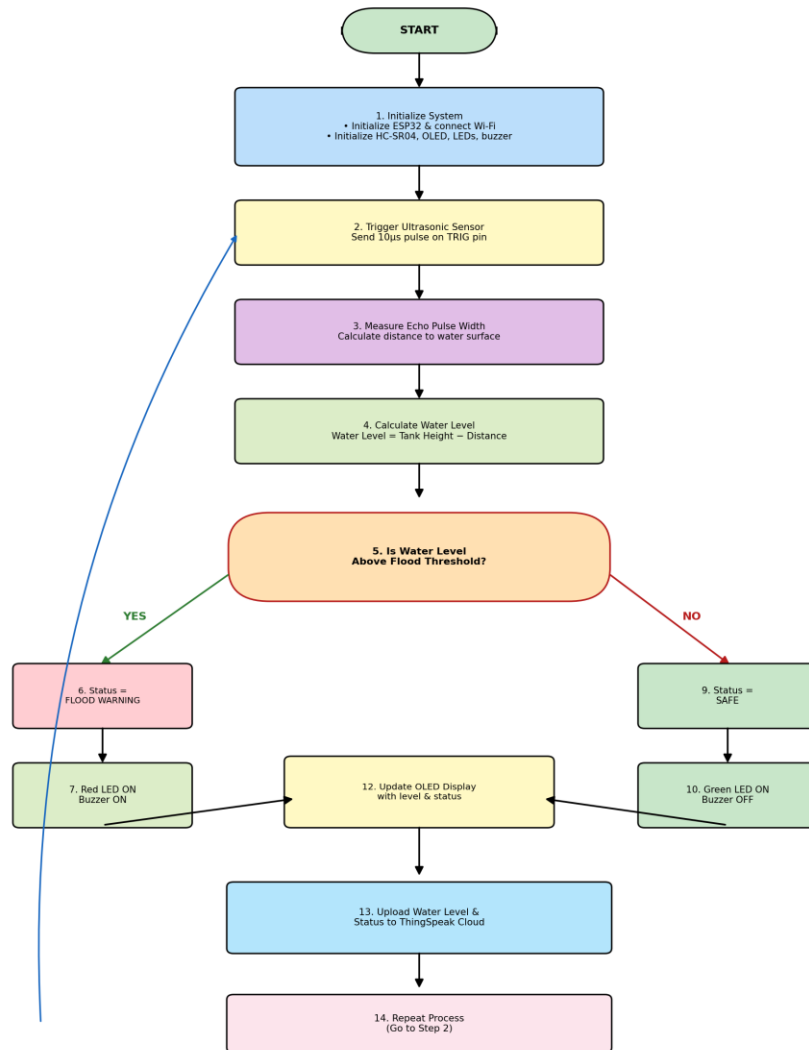


7. Algorithm

1. Start the system.
2. Initialize the ESP32 and connect to Wi-Fi.
3. Initialize the HC-SR04, OLED display, LEDs and buzzer.
4. Send a 10 μ s trigger pulse on the ultrasonic sensor's TRIG pin.
5. Measure the echo pulse width returned on the ECHO pin.
6. Calculate the distance to the water surface from the pulse width.
7. Calculate the water level as (tank/mount height – measured distance).
8. Compare the calculated water level with the predefined flood threshold.
9. If the water level is above the threshold, set Status = FLOOD WARNING, turn ON the red LED and buzzer.
10. If the water level is within the safe range, set Status = SAFE, turn ON the green LED and turn OFF the buzzer.
11. Update the OLED display with the current water level and status.
12. Upload the water level and status to the ThingSpeak cloud platform.
13. Wait for a short interval.
14. Repeat the process continuously.

SMART FLOOD LEVEL MONITORING SYSTEM

ALGORITHM FLOWCHART



8. Coding

The following Arduino program is designed for the ESP32 used in the project.

```
#include <WiFi.h>
```

```
#include <HTTPClient.h>
```

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
#define WIFI_SSID "kowsi"
```

```
#define WIFI_PASSWORD "kowsi @ 01"
```

```
// =====
```

```
// THINGSPEAK DETAILS
```

```
// =====
```

```
#define THINGSPEAK_API_KEY "BHQUMCVQN5T6PAI3"
```

```
const char* thingspeakServer =
```

```
    "http://api.thingspeak.com/update";
```

```
// =====
```

```
// PIN CONNECTIONS
```

```
// =====
```

```
// Ultrasonic sensor
```

```
#define TRIG_PIN 5
```

```
#define ECHO_PIN 18
```

```
// LEDs
```

```
#define GREEN_LED 25
```

```
#define RED_LED 26
```

```
// Buzzer
```

```
#define BUZZER 27
```

```
// LCD
```

```
#define SDA_PIN 21
```

```
#define SCL_PIN 22
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2);
```



```
// =====
```

```
// FLOOD LEVEL SETTINGS
```

```
// =====
```

```
// Distance between ultrasonic sensor and water
```

```
#define SAFE_DISTANCE 30
```

```
#define DANGER_DISTANCE 10
```

```
// =====
```

```
// THINGSPEAK TIMER
```

```
// =====
```

```
// ThingSpeak free account requires an update
```

```
// interval of at least 15 seconds.
```

```
unsigned long lastThingSpeakUpdate = 0;
```

```
const unsigned long thingSpeakInterval = 20000;
```

```
// =====
```

```
// SETUP
```

```
// =====
```

```
void setup()
```

```
{
```

```
  Serial.begin(115200);
```

```
// =====
```

```
// ULTRASONIC SENSOR
```

```
// =====
```

```
pinMode(TRIG_PIN, OUTPUT);
```

```
pinMode(ECHO_PIN, INPUT);
```

```
// =====
```

```
// LEDs
```

```
// =====
```

```
pinMode(GREEN_LED, OUTPUT);
```

```
pinMode(RED_LED, OUTPUT);
```

```
// =====
```

```
// BUZZER
```

```
// =====
```

```
pinMode(BUZZER, OUTPUT);
```

```
digitalWrite(GREEN_LED, LOW);
```

```
digitalWrite(RED_LED, LOW);
```

```
digitalWrite(BUZZER, LOW);
```

```
// =====
```

```
// LCD
```

```
// =====
```

```
Wire.begin(SDA_PIN, SCL_PIN);
```

```
lcd.init();
```

```
lcd.backlight();
```

```
// Startup message
```

```
lcd.clear();
```

```
lcd.setCursor(0, 0);
```

```
lcd.print("SMART FLOOD");
```

```
lcd.setCursor(0, 1);
```

```
lcd.print("MONITORING");
```

```
delay(2000);
```

```
lcd.clear();
```

```
// =====
```

```
// CONNECT TO WIFI
```

```
// =====
```

```
Serial.println();
```

```
Serial.println("Connecting to WiFi...");
```

```
WiFi.begin(
```

```
    WIFI_SSID,
```

```
    WIFI_PASSWORD
```

```
);
```

```
while (WiFi.status() != WL_CONNECTED)
```

```
{
```

```
Serial.print(".");
```

```
delay(500);
```

```
}
```

```
Serial.println();
```

```
Serial.println("WiFi Connected!");
```

```
Serial.print("ESP32 IP Address: ");
```

```
Serial.println(WiFi.localIP());
```

```
// =====
```

```
// LCD WIFI MESSAGE
```

```
// =====
```

```
lcd.clear();
```

```
lcd.setCursor(0, 0);
```

```
lcd.print("WiFi Connected");
```

```
lcd.setCursor(0, 1);
```

```
lcd.print("ThingSpeak Ready");
```

```
delay(2000);
```

```
lcd.clear();
```

```
Serial.println();
```

```
Serial.println("=====");
```

```
Serial.println(" SMART FLOOD MONITORING SYSTEM");
```

```
Serial.println("=====");
```

```
Serial.println("System Ready");  
}  
  
// =====  
// GET ULTRASONIC DISTANCE  
// =====  
  
float getDistance()  
{  
    // Make sure trigger is LOW  
  
    digitalWrite(TRIG_PIN, LOW);  
  
    delayMicroseconds(2);  
  
    // Send 10 microsecond pulse  
  
    digitalWrite(TRIG_PIN, HIGH);  
  
    delayMicroseconds(10);  
  
    digitalWrite(TRIG_PIN, LOW);  
  
    // Measure echo time  
  
    long duration =  
        pulseIn(ECHO_PIN, HIGH, 30000);  
  
    // Sensor timeout
```

```
if (duration == 0)
```

```
{
```

```
    return -1;
```

```
}
```

```
// Calculate distance in cm
```

```
float distance =
```

```
    duration * 0.0343 / 2;
```

```
return distance;
```

```
}
```

```
// =====
```

```
// CALCULATE WATER LEVEL
```

```
// =====
```

```
int calculateWaterLevel(float distance)
```

```
{
```

```
/*
```

```
    Maximum safe measurement = 30 cm
```

```
    Minimum measurement = 0 cm
```

```
    As distance decreases,
```

```
    water level percentage increases.
```

```
*/
```

```
int level =
```

```
    ((SAFE_DISTANCE - distance)
```

```
    / SAFE_DISTANCE) * 100;
```

```
if (level < 0)
```

```
{
```

```
    level = 0;
```

```
}
```

```
if (level > 100)
```

```
{
```

```
    level = 100;
```

```
}
```

```
return level;
```

```
}
```

```
// =====
```

```
// SEND DATA TO THINGSPEAK
```

```
// =====
```

```
void sendToThingSpeak(
```

```
    float distance,
```

```
    int waterLevel,
```

```
    int floodStatus
```

```
)
```

```
{
```

```
    // Check WiFi connection
```

```
    if (WiFi.status() != WL_CONNECTED)
```

```
    {
```

```
        Serial.println(
```

```
            "WiFi disconnected!"
```

```
);
```

```
return;
```

```
}
```

```
HTTPClient http;
```

```
// =====
```

```
// CREATE THINGSPEAK URL
```

```
// =====
```

```
String url =
```

```
String(thingspeakServer) +
```

```
"?api_key=" +
```

```
THINGSPEAK_API_KEY +
```

```
"&field1=" +
```

```
String(distance, 1) +
```

```
"&field2=" +
```

```
String(waterLevel) +
```

```
"&field3=" +
```

```
String(floodStatus);
```

```
Serial.println();
```

```
Serial.println(
```

```
"Sending data to ThingSpeak..."
```

```
);
```

```
// Start HTTP connection
```

```
http.begin(url);
```



```
// Send GET request
```

```
int httpCode =
```

```
    http.GET();
```

```
// =====
```

```
// CHECK RESPONSE
```

```
// =====
```

```
if (httpCode > 0)
```

```
{
```

```
    Serial.print(
```

```
        "ThingSpeak HTTP Code: "
```

```
    );
```

```
    Serial.println(httpCode);
```

```
if (httpCode == 200)
```

```
{
```

```
    String response =
```

```
        http.getString();
```

```
    Serial.print(
```

```
        "ThingSpeak Entry ID: "
```

```
    );
```

```
    Serial.println(response);
```

```
    Serial.println(
```

```
        "Cloud Upload: SUCCESS"
    );
}
}
else
{
    Serial.print(
        "ThingSpeak Error: "
    );

    Serial.println(httpCode);
}

// Close connection

http.end();
}

// =====
// MAIN LOOP
// =====

void loop()
{
    // =====

    // READ DISTANCE

    // =====

    float distance =
        getDistance();
```

```
// =====
```

```
// SENSOR ERROR
```

```
// =====
```

```
if (distance < 0)
```

```
{
```

```
  Serial.println(
```

```
    "Ultrasonic sensor error!"
```

```
  );
```

```
  lcd.clear();
```

```
  lcd.setCursor(0, 0);
```

```
  lcd.print("Sensor Error!");
```

```
  lcd.setCursor(0, 1);
```

```
  lcd.print("Check HC-SR04");
```

```
  digitalWrite(
```

```
    GREEN_LED,
```

```
    LOW
```

```
  );
```

```
  digitalWrite(
```

```
    RED_LED,
```

```
    LOW
```

```
  );
```

```
  digitalWrite(
```

```
BUZZER,  
LOW  
);  
  
delay(2000);  
  
return;  
}  
  
// =====  
// CALCULATE WATER LEVEL  
// =====  
  
int waterLevel =  
    calculateWaterLevel(distance);  
  
// =====  
// FLOOD STATUS  
// =====  
  
String floodStatus;  
  
// ThingSpeak numeric status  
  
int floodStatusValue;  
  
// =====  
// SAFE CONDITION  
// =====
```

```
if (distance > SAFE_DISTANCE)
```

```
{
```

```
    floodStatus = "SAFE";
```

```
    // ThingSpeak:
```

```
    // 0 = SAFE
```

```
    floodStatusValue = 0;
```

```
    // Green LED ON
```

```
    digitalWrite(
```

```
        GREEN_LED,
```

```
        HIGH
```

```
    );
```

```
    // Red LED OFF
```

```
    digitalWrite(
```

```
        RED_LED,
```

```
        LOW
```

```
    );
```

```
    // Buzzer OFF
```

```
    digitalWrite(
```

```
        BUZZER,
```

```
        LOW
```

```
    );
```

```
// =====
```

```
// LCD
```

```
// =====
```

```
lcd.clear();
```

```
lcd.setCursor(0, 0);
```

```
lcd.print("Water:");
```

```
lcd.print(distance, 1);
```

```
lcd.print("cm");
```

```
lcd.setCursor(0, 1);
```

```
lcd.print("STATUS: SAFE");
```

```
}
```

```
// =====
```

```
// WARNING CONDITION
```

```
// =====
```

```
else if (
```

```
distance > DANGER_DISTANCE &&
```

```
distance <= SAFE_DISTANCE
```

```
)
```

```
{
```

```
floodStatus = "WARNING";
```

```
// ThingSpeak:
```

```
// 1 = WARNING
```

```
floodStatusValue = 1;
```

```
// Green LED OFF
```

```
digitalWrite(  
  GREEN_LED,  
  LOW  
);
```

```
// Red LED ON
```

```
digitalWrite(  
  RED_LED,  
  HIGH  
);
```

```
// =====
```

```
// BEEPING BUZZER
```

```
// =====
```

```
digitalWrite(  
  BUZZER,  
  HIGH  
);
```

```
delay(200);
```

```
digitalWrite(  
  BUZZER,  
  LOW  
);
```

```
// =====
```

```
// LCD
```

```
// =====
```

```
lcd.clear();
```

```
lcd.setCursor(0, 0);
```

```
lcd.print("Water:");
```

```
lcd.print(distance, 1);
```

```
lcd.print("cm");
```

```
lcd.setCursor(0, 1);
```

```
lcd.print("WARNING!");
```

```
}
```

```
// =====
```

```
// FLOOD CONDITION
```

```
// =====
```

```
else
```

```
{
```

```
  floodStatus = "FLOOD";
```

```
  // ThingSpeak:
```

```
  // 2 = FLOOD
```

```
  floodStatusValue = 2;
```

```
  // Green LED OFF
```



```
digitalWrite(  
  GREEN_LED,  
  LOW  
);
```

```
// Red LED ON
```

```
digitalWrite(  
  RED_LED,  
  HIGH  
);
```

```
// Continuous buzzer
```

```
digitalWrite(  
  BUZZER,  
  HIGH  
);
```

```
// =====
```

```
// LCD
```

```
// =====
```

```
lcd.clear();
```

```
lcd.setCursor(0, 0);
```

```
lcd.print("Water:");
```

```
lcd.print(distance, 1);
```

```
lcd.print("cm");
```

```
    lcd.setCursor(0, 1);  
    lcd.print("!!! FLOOD !!!");  
}
```

```
// =====  
// SERIAL MONITOR  
// =====
```

```
Serial.println();  
Serial.println(  
    "-----"  
);
```

```
Serial.print(  
    "Distance: "  
);
```

```
Serial.print(  
    distance,  
    1  
);
```

```
Serial.println(  
    " cm"  
);
```

```
Serial.print(  
    "Water Level: "  
);
```

```
Serial.print(  
  waterLevel  
);
```

```
Serial.println(  
  " %"  
);
```

```
Serial.print(  
  "Status: "  
);
```

```
Serial.println(  
  floodStatus  
);
```

```
// =====  
// THINGSPEAK UPDATE  
// =====
```

```
if (  
  millis() - lastThingSpeakUpdate >=  
  thingSpeakInterval  
)  
{  
  lastThingSpeakUpdate =  
    millis();  
  
  sendToThingSpeak(  
    "
```

```

        distance,
        waterLevel,
        floodStatusValue
    );
}

Serial.println(
    "-----"
);
delay(5000);
}

```

Code Explanation

- GPIO 5 triggers the HC-SR04 ultrasonic pulse; GPIO 18 reads the returning echo.
- GPIO 2 controls the green (safe) LED; GPIO 4 controls the red (flood warning) LED.
- GPIO 15 controls the buzzer.
- GPIO 21/22 (I2C) drive the OLED display through the Adafruit_SSD1306 library.
- mountHeight defines the sensor's fixed height above the tank/canal bed, used to convert distance into water level.
- floodThreshold defines the water level above which the system reports a flood warning.
- ThingSpeak.setField() and ThingSpeak.writeFields() upload the water level and flood status to the cloud channel.

The mountHeight and floodThreshold values should be calibrated according to the actual sensor mounting height and the flood-risk level appropriate to the monitored site.

9. System Working

When the system is powered ON, the ESP32 initializes the ultrasonic sensor, OLED display, LEDs and buzzer, and connects to the Wi-Fi network.

The HC-SR04 continuously measures the distance to the water surface, which the ESP32 converts into a water-level reading.

Safe Condition

When the water level remains below the flood threshold:

Water Level Normal

↓

ESP32 detects safe condition

↓
Green LED ON, Buzzer OFF

↓
Status: SAFE

Flood Warning Condition

When the water level rises above the flood threshold:

Water Level High

↓
ESP32 detects flood condition

↓
Red LED ON, Buzzer ON

↓
Status: FLOOD WARNING

At the same time, the ESP32 updates the OLED display and uploads the water level and status to ThingSpeak, allowing the condition to be monitored remotely by disaster-management personnel. This process repeats automatically, providing continuous flood-level monitoring.

10. Bill of Materials

S.No.	Component	Quantity
1	ESP32 Development Board	1
2	HC-SR04 Ultrasonic Sensor	1
3	OLED Display (I2C, 128x64)	1
4	Green LED	1
5	Red LED	1
6	Buzzer	1
7	Breadboard	1
8	Jumper Wires	As required
9	Regulated Power Supply	1

Software Requirements

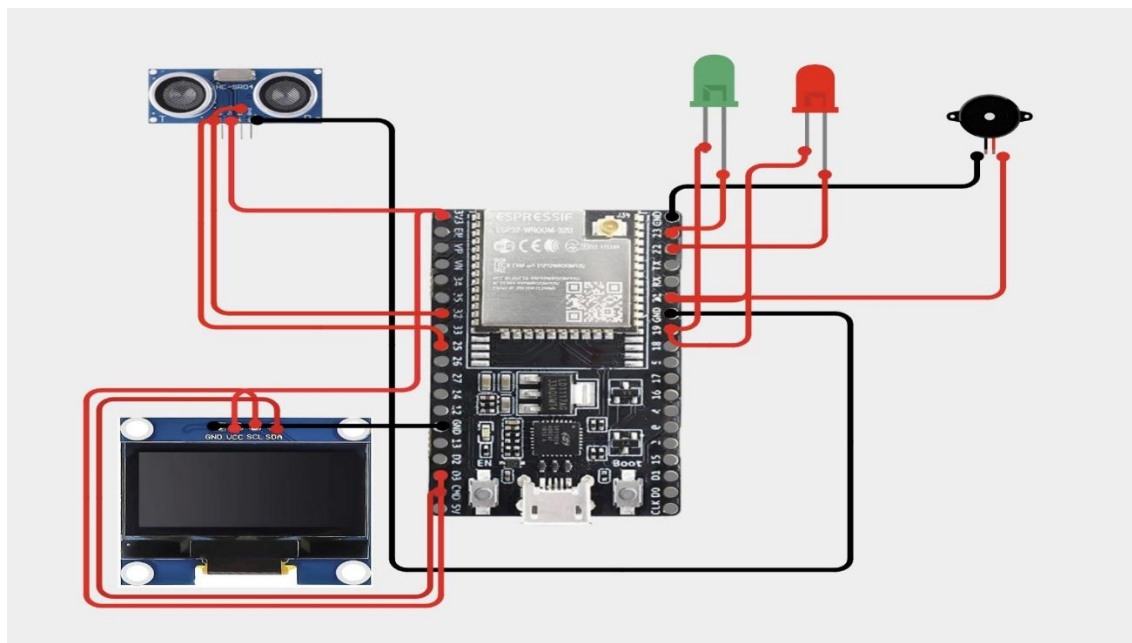
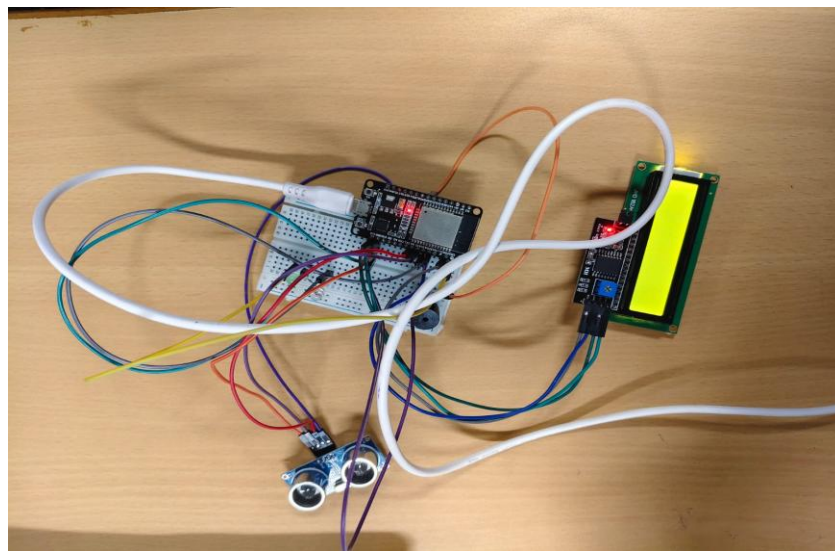
- Arduino IDE
- ESP32 Board Package
- Adafruit GFX Library
- Adafruit SSD1306 Library
- ThingSpeak Arduino Library

11. Prototype Implementation

The prototype was assembled using an ESP32 development board, an HC-SR04 ultrasonic sensor, an OLED display, a green LED, a red LED, and a buzzer, wired on a breadboard as per the circuit table above.

The ESP32 was selected as the main controller because it provides sufficient GPIO pins for the sensor, indicators and display, along with built-in Wi-Fi connectivity for direct IoT cloud integration without an additional communication module.

The prototype continuously measures the distance to the water surface, calculates the water level, drives the LED and buzzer alerts accordingly, and streams the readings to the OLED display and the ThingSpeak channel.



12. Results & Performance Analysis

Expected Results

The system successfully demonstrates continuous, automated flood-level monitoring with immediate local alerting and remote cloud visibility.

Condition	Sensor Condition	LED	Buzzer	Status
Normal water level	Level below threshold	Green ON	OFF	SAFE
Rising / flood-level water	Level above threshold	Red ON	ON	FLOOD WARNING

Performance

The system provides the following benefits:

- Real-time, non-contact water-level monitoring.
- Early detection of rising flood conditions.
- Immediate local alerting through LEDs and buzzer.
- Remote monitoring and historical data logging through ThingSpeak.
- Reduced dependency on manual site inspection.
- Low-cost and easily deployable at multiple flood-prone locations.
- Supports faster, data-driven disaster-management response.

Applications

- River and canal flood monitoring
- Dam and reservoir water-level tracking
- Urban stormwater drain monitoring
- Agricultural field flood detection
- Early-warning systems for disaster management authorities

Conclusion

The Smart Flood Level Monitoring System provides a low-cost and effective approach to continuous water-level monitoring for disaster management applications. By using an ultrasonic sensor to measure the water level in a non-contact manner, the system can detect rising flood conditions and alert nearby personnel immediately through LED indicators and a buzzer, while also logging the data to the cloud for remote monitoring.

The ESP32 acts as the central controller, the HC-SR04 provides the water-level measurement, the OLED display provides local readout, and ThingSpeak provides remote, cloud-based monitoring for disaster-management authorities.

The proposed system helps shift flood monitoring from manual, periodic inspection to continuous, automated observation, enabling faster warning and response. In future development, additional features such as rainfall sensors, multi-point sensor networks, SMS/mobile-app alerts, solar-powered operation, and machine-learning-based flood prediction can be incorporated.